Arbeitsschritte der Programmerstellung:

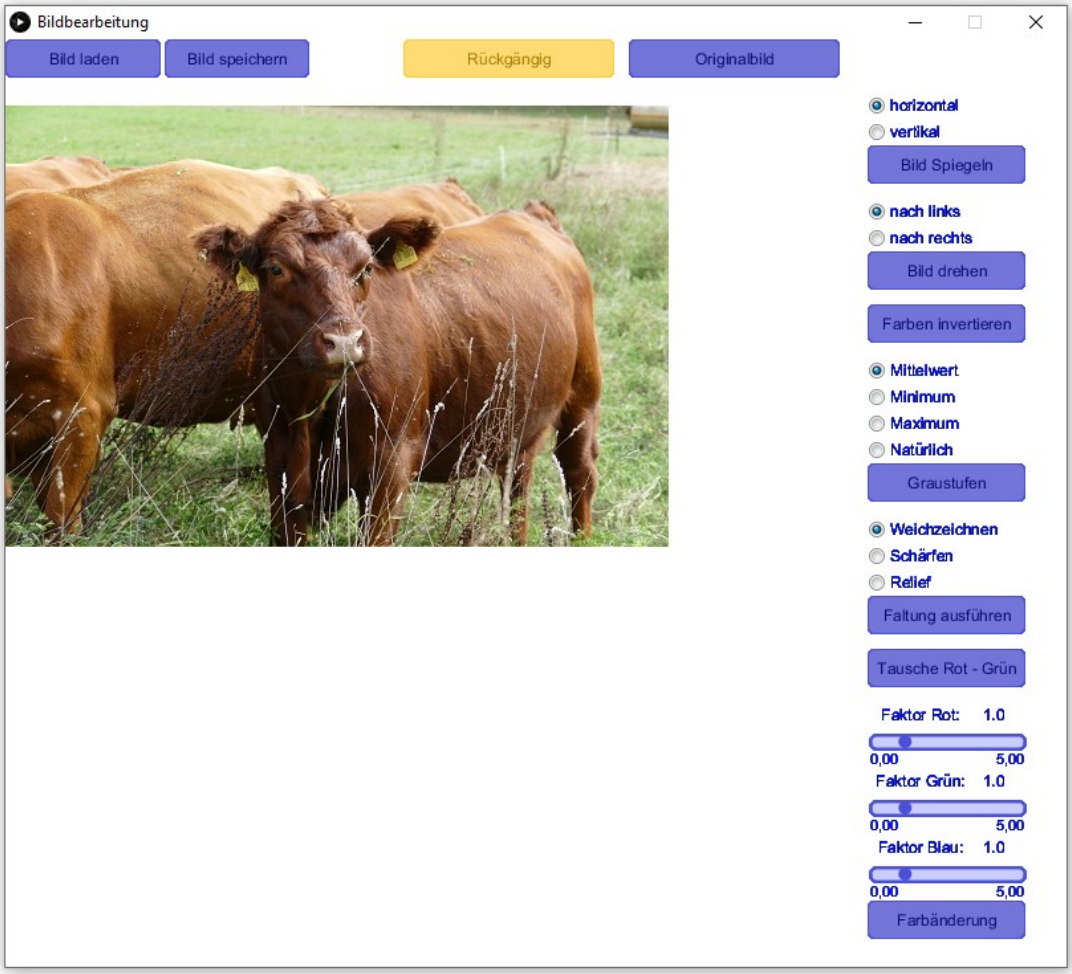
Nachdem die Algorithmen zur Bildbearbeitung implementiert und getestet wurden, kann nun die Programmoberfläche (Graphical User Interface) erstellt werden und mit den Algorithmen verknüpft werden. Daraus ergeben sich die drei Schritte:

1. Implementation und Testen der Algorithmen (**Model**)
2. Gestalten der Oberfläche mit einem GUI-Editor: hier G4P GUI Builder[1] (**View**)
3. Verknüpfen der GUI mit der Oberfläche: Implementation von Aktionen (**Controller**)

Die Trennung eines Programmierprojekts in diese drei Teile nennt man auch **Model-View-Controller (MVC)** Konzept und ist für größere Projekte Pflicht, da es die Übersicht erhöht. Die Interaktion zwischen den drei Teilen vereinfachen wir aber hier in der Schule gegenüber dem "echten" MVC-Konzept.

Gestaltung der Oberfläche

Am Ende könnte die Oberfläche in etwa so aussehen[2]:



1 In Processing: Hinzufügen über Tools – Tool hinzufügen: G4P GUI Builder auswählen und installieren

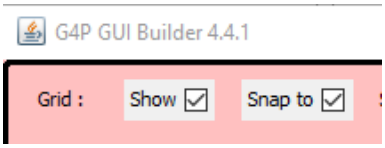
2 Alle Screenshots von eigenen Processing-Programmen, alle Bilder privat (Eisenmann)



Der G4P GUI Builder, der als Tool in Processing eingebunden werden kann, ist sehr einfach gehalten und ist deshalb auch einfach zu bedienen.

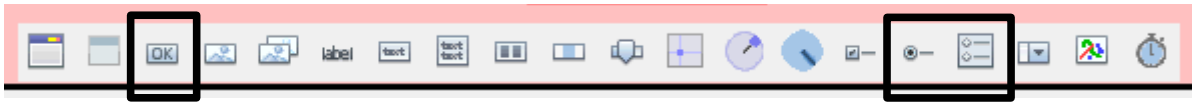
Erstellen der GUI

1. *Starte eine neue Processing-Datei und speichere sie unter „bildbearbeitung_v1“ ab.*
2. *Starte dann unter Tools den G4P GUI Builder.*
3. *In deinem Processing-Fenster wurden einige Zeilen Code automatisch eingefügt. Ändere in der setup()-Methode die Größe des Fensters auf 800 x 700 Pixel und lösche – sollte es in deiner Processing-Version vorhanden sein – den Import von „peasy“. Alles andere lässt du zunächst unverändert.*
4. *Stelle in G4P zunächst einmal das Gitter ein und lasse alles, was du einfügen wirst, am Gitter andocken.*



Für unsere Oberfläche brauchen wir Schaltflächen (Buttons) und die Möglichkeit, eine Auswahl zu treffen, um z.B. vor dem Klick auf eine Schaltfläche „Spiegeln“ auswählen zu können, ob horizontal oder vertikal gespiegelt werden soll. Für jede Auswahl nutzen wir eine Gruppe (ToggleGroup) mit mehreren Auswahlfeldern (Options). Die Gruppe regelt, dass immer nur eine Auswahl getroffen sein kann[3].

Eine Komponente wird einfach oben in der Leiste angeklickt und erscheint dann in der Mitte des Fensters. Von dort aus lässt sie sich mit der Maus an die gewünschte Stelle ziehen.



Ist eine Komponente ausgewählt, so kann man rechts unter „Properties“, wie der Name schon sagt, verschiedenen Eigenschaften der Komponente Werte zuweisen. Bei den Schaltflächen verändern wir den Namen der Komponente (Variable name), den Namen der Ereignismethode (Event method) und die Aufschrift (Text).

PROPERTIES	
GUI LOCK	☐
Variable Name	btSpiegeln
Event method	btSpiegelnClick
Text	Bild spiegeln

Zum Beispiel für die Schaltfläche zum Spiegeln des Bildes: „btSpiegeln“, „btSpiegelnClick“ und „Bild spiegeln“. Das „bt“ (oder auch „btn“) zu Beginn des Namens steht für die Art der Komponente, den Button. Das kennst du evtl. schon aus Klasse 8 vom MIT App Inventor. Das macht es beim Programmieren leichter zu erkennen, was sich hinter dem Namen versteckt.

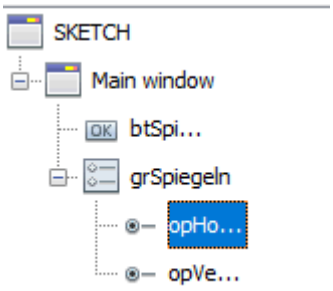
PROPERTIES	
GUI LOCK	☐
Variable Name	opHorizontal
Event method	
Selected?	☑
Display text	horizontal

Die Gruppe für die Auswahl nennen wir grSpiegeln („gr“ für group), die beiden zur Gruppe gehörenden Auswahl-Optionen nennen wir „opHorizontal“ und „opVertikal“. Da beim Klick auf eine Option, diese nur ausgewählt werden soll, aber ansonsten nichts passieren muss, leeren wir das „Event method“-Feld bei beiden und verändern nur den Text (Display text) zu „horizontal“ und „vertikal“.

3 Möchtest du noch Informationen über weitere Komponenten, kannst du auf der Seite des Programmieres Informationen und kleine Videosequenzen finden: *http://lagers.org.uk/g4ptool/index.html* (abgerufen am: 26.06.2020)

Die Komponenten sind auf dem Fenster sichtbar und werden gleichzeitig rechts hierarchisch aufgelistet:





5. *Gestalte deine Oberfläche mit Schaltflächen und Auswahlmöglichkeiten für alle Bildbearbeitungsarten, die du bisher schon programmiert hast. Achte dabei auf eine sinnvolle Namensgebung, wie oben beschrieben.*
6. *Starte dein Programm und schaue dir dein Werk an.*

Bevor wir uns jetzt um die Ereignismethoden kümmern können, brauchen wir die ganzen Bildbearbeitungsmethoden, die wir schon geschrieben haben.

7. *Kopiere aus deinen bisherigen Processing-Programmen alles Nötige für das Spiegeln des Bildes in das neue Programm mit GUI.*

Hier siehst du, wo dein Quelltext ergänzt werden muss:

bildbearbeitung_v1 | gui

```
1  // Need G4P library
2  import g4p_controls.*;
3
4  // ********* Deklarationen
5
6  public void setup(){
7    size(800, 700, JAVA2D);
8    createGUI();
9    customGUI();
10
11   // ********** setup()-Methode ergänzen, aber hier keine Bildbearbeitungs-
12   //            methode aufrufen!
13 }
14
15 public void draw(){
16   // background(230);  // wird hier auskommentiert, kommt in setup()
17 }
18
19 // ************* hier alle Bildbearbeitungsmethoden einfügen
20
21
22 // Use this method to add additional statements
23 // to customise the GUI controls
24 public void customGUI(){
25 }
```



Jetzt müssen wir noch die GUI mit den Aktionen verknüpfen. Indem wir beim Erstellen der GUI den Schaltflächen Ereignismethoden, wie z.B. btSpiegelnClick, zugewiesen haben, ist der erste Schritt schon erledigt: die Schaltfläche „btSpiegeln“ wartet ständig darauf, dass auf sie geklickt wird.

Wenn das der Fall ist, wird das ausgeführt, was in der Ereignisbehandlungsmethode „btSpiegelnClick“ geschrieben wurde. Um das müssen wir uns jetzt noch kümmern.

Den Rahmen dieser Methoden finden wir in dem automatisch mit der GUI erstellten zweiten Quelltext.

Achtung: Dort füllen wir nur genau diese Ereignismethoden. Alles andere schreiben wir weiterhin im Quelltext „bildbearbeitung_v1“.

Folgende Zeilen wurden automatisch eingefügt:

```
16
17 public void btSpiegelnClick(GButton source, GEvent event) { //_CODE_:btSpiegeln:781683:
18   println("btBildLaden - GButton >> GEvent." + event + " @ " + millis());
19 } //_CODE_:btSpiegeln:781683:
```

Beim Klick auf den Button wird bisher eine Information in der Konsolenzeile ausgegeben.

8. *Starte dein Programm, wie es bisher ist und klicke auf den Spiegeln-Button. Achte dabei auf die Ausgabe in der Konsole.*
9. *Kommentiere jetzt diese Zeile aus (oder lösche sie).*
10. *Überlege dir, was passieren soll, wenn auf den Button geklickt wird und formuliere es mit eigenen Worten.*

Falls „horizontal“ ausgewählt ist, dann soll das Bild horizontal gespiegelt werden, sonst vertikal.

Anschließend müssen wir das veränderte Bild wieder anzeigen, wie wir es bisher auch schon gemacht haben.

Die Methode könnte folgendermaßen aussehen:

```
public void btSpiegelnClick(GButton source, GEvent event) { //_CODE_:btSpiegeln:
  if (opHorizontal.isSelected()) {
    bildNeu = spiegleHorizontal(bildAlt);
  } else {
    bildNeu = spiegleVertikal(bildAlt);
  }
  background(255); // altes Bild wird übermalt
  image(bildNeu, 0, 0); // Bild wird an der Position (0|0) angezeigt
} //_CODE_:btSpiegeln:781683:
```

11. *Teste das Programm.*
12. *Ergänze eine Bildbearbeitungsart nach der anderen. Und mache erst weiter, wenn du getestet hast und mit dem Ergebnis zufrieden bist.*



Zusatzaufgaben:

1. *Informiere dich, wie ein sogenannter Slider funktioniert und baue für die drei Farben rot, grün und blau einen Slider ein, mit dem man zur Laufzeit deines Programmes die Farbanteile verändern kann. Nutze dann die Methode „farbaenderung()“.*
2. *Verändere dein Programm so, dass man ein neues Bild laden kann.*
3. *Verändere dein Programm so, dass man ein verändertes Bild speichern kann.*
4. *Schaffe eine Möglichkeit, eine Aktion rückgängig machen zu können.*
5. *Hole dir ein Arbeitsblatt zu einer weiteren Bildbearbeitungsmöglichkeit (Erweiterungen), die dir gefällt und bearbeite die Aufgaben.*